Project proposal and first glance

Track 1, energy: grid load demand (PJM East)

Author: Xin Guo

ClassNo: DSE6230OM

1. Track selection and rationale

I'm taking Track 1, forecasting hourly electricity load for the PJM East zone.

- **Hourly load carries three real seasonal cycles: daily, weekly, and annual.** That gives every method in this course something to work with, so ETS, ARIMA, and a lag-based gradient boosting model can be compared on the same backtest without any of them being a strawman.
- **The business side is just as concrete.** PJM schedules generation a day ahead, and forecast error costs money in a direction you can name. Too low and the utility buys expensive peaking power. Too high and it pays for capacity that sits idle.
- **Target and horizon.** Hourly load in megawatts (PJME_MW), forecast 24 hours ahead: at a fixed cut-off each day, produce the next 24 hourly values. I fixed the horizon before anything else, because it decides how the model is scored and how a lag model has to generate its own inputs past the cut-off.
- **Code.** I'm building this on a config-driven forecasting pipeline I already run end to end: https://github.com/VickyGuo0907/ml_pipeline. Also, I included a reference notebook for reference as well under notebooks/pjm_load_forecast_pipeline.ipynb, and the stage configuration is config/pjm_load_forecast/. Section 4 reports what it produces today.

2. Data access confirmation (first glance)

The source is the PJM Interconnection Hourly Energy Consumption dataset on Kaggle (robikscube), file PJME_hourly.csv. It loads with no parsing errors and needs no cleaning to inspect.

```
import pandas as pd

df = pd.read_csv('sample_data/PJME_hourly.csv', parse_dates=['Datetime'])

df.head()
```

	Datetime	PJME_MW
0	2002-12-31 01:00:00	26498.0
1	2002-12-31 02:00:00	25147.0
2	2002-12-31 03:00:00	24574.0
3	2002-12-31 04:00:00	24393.0
4	2002-12-31 05:00:00	24860.0

The file isn't sorted by time. Those first rows are from December 2002, so sorting is the first step in the cleaning stage.

Shape, coverage, and frequency

```
pd.DataFrame(
    [
        ("Rows x columns", f"{glance_df.shape[0]:,} x {glance_df.shape[1]}"),
        ("Columns", ", ".join(f"{c} ({glance_df[c].dtype})" for c in glance_df.columns)),
        ("Date range", f"{glance_df.Datetime.min()} to {glance_df.Datetime.max()}"),
        ("Native frequency", pd.infer_freq(grid.index)),
        ("Number of series", "1 (PJME zone, aggregated)"),
        ("Unique timestamps", f"{glance_df.Datetime.nunique():,} (expected {expected:,})"),
        ("Missing hours", int(grid.isna().sum())),
        ("Duplicated timestamps", int(glance_df.Datetime.duplicated().sum())),
        ("Null values", glance_df.isna().sum().to_dict()),
        ("Load statistics", f"mean {filled.mean():,.0f} | SD {filled.std():,.0f} | "
                            f"min {filled.min():,.0f} | max {filled.max():,.0f}"),
    ],
    columns=["Property", "Value"],
).set_index("Property")
```

Property	Value
Rows x columns	145,366 x 2
Columns	Datetime (datetime64[ns]), PJME_MW (float64)
Date range	2002-01-01 01:00:00 to 2018-08-03 00:00:00
Native frequency	h
Number of series	1 (PJME zone, aggregated)
Unique timestamps	145,362 (expected 145,392)
Missing hours	30
Duplicated timestamps	4
Null values	{'Datetime': 0, 'PJME_MW': 0}
Load statistics	mean 32,079 | SD 6,464 | min 14,544 | max 62,009

The 30 gaps and 4 duplicates both come from daylight saving. The spring-forward hour never happens, and the fall-back hour gets recorded twice. That's 0.02% of rows, so it's bookkeeping rather than a data quality problem, but I have to handle it before building a single lag feature. It's covered in section 5 below.

3. Initial EDA

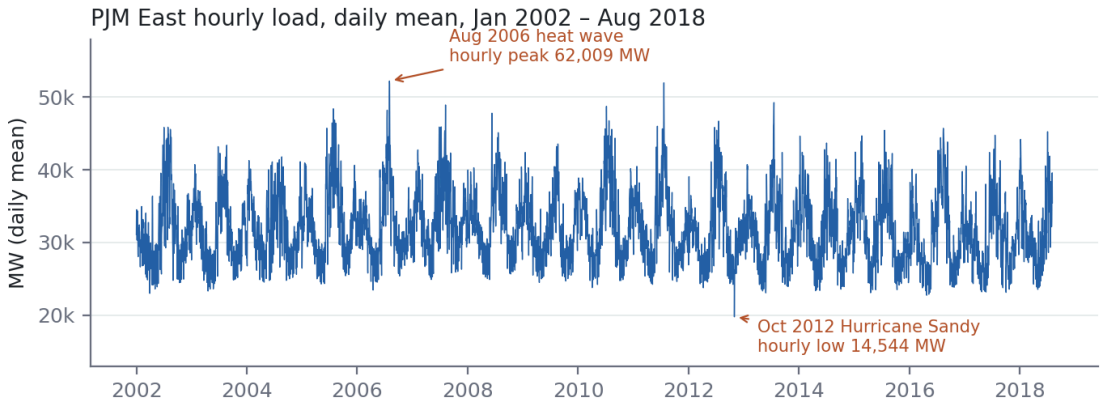


Figure 1. Daily mean load across the full history. Both annotated points are real events, not data errors.

- **Pattern 1: a bimodal annual cycle with a flat level.** Load peaks twice a year. Summer is the big one, driven by air conditioning, with July averaging 37,882 MW. Winter heating gives a smaller peak at 34,343 MW in January; the mild months sag, with April down at 27,863 MW.

  The level barely moves across 16 years. Annual mean load was 31,566 MW in 2002, and 30,652 MW in 2017, so efficiency gains have roughly cancelled out demand growth. A trend component has almost nothing to do here, and a model that extrapolates one aggressively will hurt me.

- **Pattern 2: outliers that are events.** The maximum, 62,009 MW at 17:00 on 2 August 2006, was a heat wave. The minimum, 14,544 MW at 04:00 on 30 October 2012, was Hurricane Sandy cutting power across a large share of the zone. I'm deleting neither. The Sandy week is demand that physically couldn't be served, and it's my best candidate for an event flag.

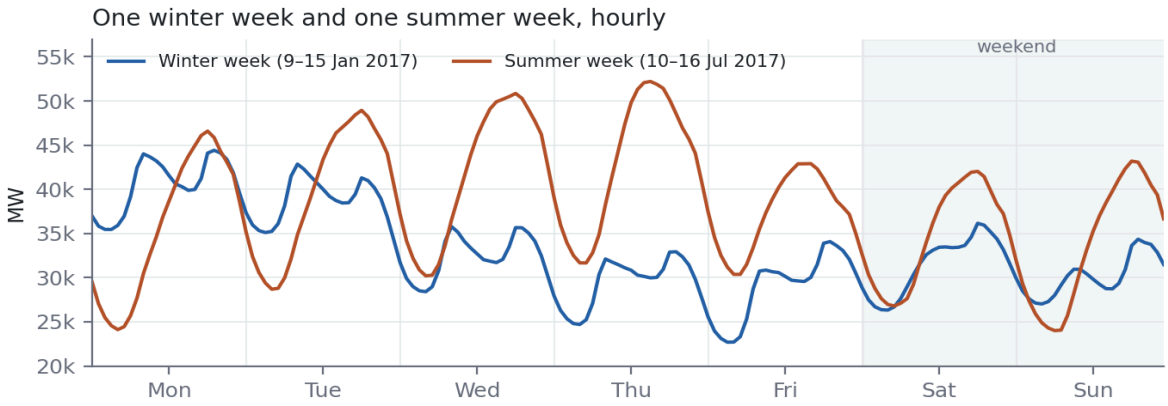


Figure 2. One winter week and one summer week, hourly. The weekend band is shaded.

- **Pattern 3: a daily cycle that changes shape by season.** Every day dips in the early morning and peaks later, but the shape isn't constant. January runs twin-peaked, with a morning rise and an evening peak at 19:00 averaging 38,939 MW. July is single-peaked and sharper, cresting at 17:00 at 45,939 MW as cooling load builds through the afternoon. A model with one fixed daily shape will miss the summer afternoon peak, which is the hour the utility cares about most.
- **Pattern 4: a weekly cycle.** Weekdays average 32,996 MW against 29,784 MW on weekends, roughly a 10% drop, and the weekend profile is flatter as well as lower.

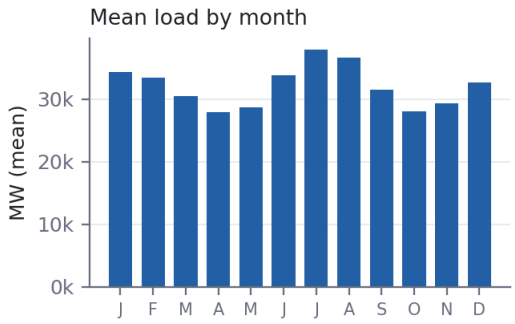


Figure 3. Mean load by month, the bimodal annual cycle.

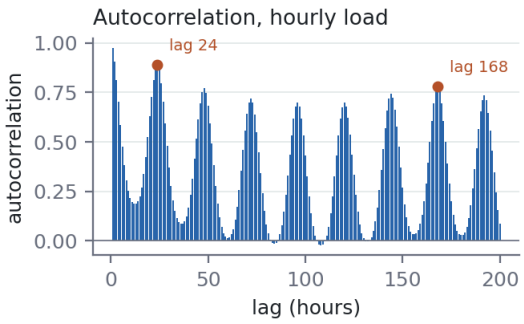


Figure 4. Autocorrelation to 200 lags, with spikes at 24 and 168 hours.

The autocorrelation function confirms it. Correlation is 0.975 at lag 1, 0.891 at lag 24, 0.782 at lag 168, and still 0.687 at lag 8,760 (one year). All three seasonal periods show up in the series itself, which is direct evidence that lag features will carry most of the signal.

4. Modeling plan

The split is chronological: 80% train and the most recent 20% test. After dropping 168 warm-up rows for the longest lag, that gives 116,179 training hours (2002-01-08 to 2015-04-10) and 29,045 test hours (2015-04-10 to 2018-08-03).

Baselines on that test window

Benchmark	MAE (MW)	RMSE (MW)	MAPE
Seasonal naive, same hour yesterday (lag 24)	2,184	3,009	6.93%
Seasonal naive, same hour last week (lag 168)	3,392	4,670	10.56%

Anything that can't beat 6.93% MAPE isn't worth reporting. The daily lag beats the weekly one by a wide margin, which says the daily cycle dominates and sets the feature order below. These two baselines currently live outside the pipeline, and folding them in as a naive model type is the first thing on my list.

Model Sequence

- Seasonal naive at lag 24 and lag 168, as a pipeline model type rather than a side calculation. Every model below is judged against it.
- ETS (Holt-Winters) with a damped trend, fitted on an MSTL-deseasonalised hourly series. Standard ETS holds one seasonal period, and hourly load has three, so a single 24-hour seasonal term leaves the weekly and annual cycles unmodelled. That limitation is a finding worth reporting, not a detail to hide.
- SARIMAX with Fourier terms for the weekly and annual periods, differencing chosen from ACF and PACF plus ADF and KPSS. This is the assumption-checkable comparison against the machine learning model.
- LightGBM on lag, rolling, and calendar features. Features are added in groups and each group has to earn its place on the backtest: lag 24, then lag 168, then rolling statistics, then calendar, then holiday flags.
- Orders and hyperparameters are set by hand, not searched. Automated tuning is out of scope for this project, so model comparison stays about structure rather than about who tuned harder.
- Recursive versus direct multi-step over the 24-hour horizon, compared directly, since error accumulation is one of the things a fixed horizon should expose. If time allows, a residual hybrid: a classical baseline with a boosted model fitted to its residuals.

Evaluation design

- Rolling-origin backtesting with a fixed 24-hour horizon and multiple origins, never a shuffled split. Expanding window, with refit cadence recorded so the results are reproducible.
- Metrics: MAE and MAPE for interpretability with an operations audience, MASE against the seasonal-naive baseline for scale-free comparison, and RMSE reported alongside so large-error behaviour is visible.
- Prediction intervals with empirical coverage checked against the nominal level — an interval nobody verified is not a risk estimate.

- Residual diagnostics: ACF and Ljung-Box on the residuals of every model, to show whether structure remains unmodelled.
- Error decomposed by hour of day and by season, because a utility cares far more about the 17:00 summer peak than about the 04:00 trough.
- Add to the metrics bullet: the champion is chosen on cross-validated MAPE, and any model that fails to beat seasonal naive is reported as a negative result with its diagnostics.
- Add a line on comparability: all families must be scored at the same origins with the same horizon anchoring, so the numbers are actually comparable.

Track technical requirements addressed

Requirement	How I address it
Multiple seasonality (daily, weekly, annual)	MSTL decomposition at 24, 168, and 8,766 hours in the profile stage, lag-24 and lag-168 features plus calendar variables for GBM, and Fourier terms in the SARIMAX rebuild
Classical against machine learning	ETS and SARIMAX against LightGBM on one shared set of rolling origins
Honest backtesting	Chronological split, rolling origins at a fixed 24-hour horizon, and recursive generation of lag features past the cut-off
Baseline comparison	Seasonal naive at lag 24 and lag 168, moving into the pipeline as a model type
Uncertainty	Prediction intervals with measured coverage
Operational framing	Error reported by hour and season, with the asymmetric cost of under- against over-forecasting discussed in the memo

5. Open risks

- No weather data, and weather is the largest driver of load. The file has two columns, so the model can only infer temperature indirectly through lagged load. That holds up reasonably at a 24-hour horizon, since yesterday's weather correlates strongly with today's, but it will fail exactly when it matters most: the first hot day of a heat wave, where lagged load says mild and reality says peak. I state the limitation plainly, and if time allows I join NOAA daily temperature for a Philadelphia-area station as an exogenous regressor and measure what it buys.
- SARIMAX is currently unusable, at roughly 100% MAPE in both CV and test. That is predictions collapsing toward zero, not a weak model, so one of my three families is dead weight today. The rebuild is Fourier terms for the weekly and annual periods in place of a lone 24-hour seasonal order, differencing chosen from the diagnostics, and a check on whether the memory-driven 2-year training cap is starving the fit. If it still fails, it gets reported as a documented negative result with the diagnostics behind it.
- Runtime and memory, both measured rather than guessed. MSTL takes 8 to 9 minutes on the full 145k rows, and SARIMAX on the full training set peaked at 11GB RSS and was OOM-killed on a 7.75GB Docker host shared by 8 services. Both are handled today: SARIMAX trains on a trailing 2-year window with simple differencing while ETS and GBM use the full series. The live risk is that prediction intervals and per-hour error analysis push runtime up again, so I measure each addition before keeping it.

- Remaining scope against the calendar. A naive baseline model type, three more metrics, interval coverage, the SARIMAX rebuild, an MSTL-based ETS refit and horizon-anchored scoring for the classical models is more than a comfortable four weeks. Priority order is baseline and metrics first, since they make every later comparison meaningful, then the SARIMAX and ETS work. Anything that misses its slot gets reported as attempted with what I learned, so nothing disappears quietly.

6. Next steps

- Week 1: seasonal naive as a pipeline model type, plus MAE, RMSE, and MASE in the evaluation report, so every model is scored the same way against the same benchmark.
- Week 2: stationarity diagnostics and the SARIMAX rebuild with Fourier terms, and ETS refitted on a deseasonalised series.
- Week 3: prediction intervals with empirical coverage checks, then the feature ladder run in groups so each addition earns its place. The ladder is the first thing I drop if week 2 runs long, since nothing else depends on its result.
- Week 4: one clean end-to-end run to produce the final numbers, then error analysis by hour and season, then the memo for a grid operations audience.